Evaluating Soft Core RISC-V Processor in SRAM-Based FPGA Under Radiation Effects

Ádria B. de Oliveira, Lucas A. Tambara, Fabio Benevenuti, Luis A. C. Benites, Nemitala Added, Vitor A. P. Aguiar, Nilberto H. Medina, Marcilei A. G. Silveira, and Fernanda L. Kastensmidt

***Abstract*— This article evaluates the RISC-V Rocket processor embedded in a Commercial Off-The-Shelf (COTS) SRAM-based field-programmable gate array (FPGA) under heavy-ions-induced faults and emulation fault injection. We also analyze the efficiency of using mitigation techniques based on hardware redundancy and scrubbing. Results demonstrated an improvement of 3× in the cross section when scrubbing and coarse grain triple modular redundancy are used. The Rocket processor presented analogous sensitivity to radiation effects as the state-of-the-art soft processors. Due to the complexity of the system-on-chip, not only the Rocket core but also its peripherals should be protected with proper solutions. Such solutions should address the specific vulnerabilities of each component to improve the overall system reliability while maintaining the trade-off with performance.**

***Index Terms*— Commercial Off-The-Shelf (COTS) SRAM-based field-programmable gate array (FPGA), fault injection (FI), fault tolerance, heavy ion, RISC-V, rocket.**

I. Introduction

SOFT CORES are synthesizable hardware models of processors commonly implemented in field-programmable gate arrays (FPGAs). These processors offer many benefits to space systems compared to application-specific integrated circuits (ASICs). Reconfiguration is a primary benefit of soft cores since their characteristics can change on-orbit. Besides, soft cores allow the evaluation of the trade-offs in resource utilization and architecture, performance, power consumption, and cost. Soft cores also provide flexibility to address changing design requirements during the development process.

Several soft processors have been developed in the last decades. They implement different instruction set architectures (ISAs) and differ in complexity, ranging from simple microcontroller implementations to complex multi-core systems-on-chip (SoCs). Some examples of soft processors currently available at academic level, that is, free of charge, are: LEON3 and LEON4 cores (SPARC ISA) from Cobham Gaisler [1]; Cortex-M0, Cortex-M1, and Cortex-M3 (RISC ISA) cores from ARM [2]; MicroBlaze (RISC ISA) from Xilinx [3]; and a plethora of soft cores that implement the RISC-V ISA, such as the Rocket core [4]. The RISC-V cores have recently emerged as processors suitable for many computing-intensive applications due to their performance-enabler architecture and reduced complexity of the instructions set, which allows designers to customize both processor cores and compilers.

In the context of applications that require a certain level of reliability, such as space, one must evaluate a soft core-based system not only in terms of area, performance, and power but also its behavior in the presence of faults. Soft core processors synthesized into Commercial Off-The-Shelf (COTS) SRAM-based FPGAs are susceptible to radiation-induced single-event effects (SEEs) at device and design levels. At device level, single-event upsets (SEUs) may occur in the configuration memory of the device and in its embedded memories [5]. SEUs in the configuration memory that implements a soft core processor can be persistent and lead to changes in the architectural implementation of the core that can cause control-flow errors, such as system hangs [single-event functional interrupt (SEFI)] and wrong computations. These faults can be corrected by reconfiguration. Concerning embedded memories and SEEs at design level, SEUs and single-event transients (SETs) can have complex effects in the processor architecture and its executing software [6]. Such effects affect processors by modifying values stored in memory elements, that is, data-flow errors, leading the processor to execute computations incorrectly [silent data corruption (SDC)]. In general, these events can be cleared out by resetting the processor.

This article aims at advancing the knowledge of radiation effects in soft cores implemented in COTS FPGAs by a reliability and performance analysis of a RISC-V processor synthesized in an SRAM-based FPGA under SEEs. We explore the FPGA configuration memory susceptibility to SEEs through fault injection (FI) by emulation and heavy ion testing. The case-study scenario consists of a Rocket processor [4] embedded in a Xilinx Zynq-7000 SoC. The main contributions of this work lie in 1) evaluating the vulnerability to permanent faults in the configuration memory of a RISC-V processor

Manuscript received March 18, 2020; revised April 26, 2020; accepted May 11, 2020. Date of publication May 19, 2020; date of current version July 16, 2020. This work was supported by Brazilian Funding Agencies: Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES)—Finance Code 001, Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), and FAPESP Proc.2012/03383-5.

Ádria B. de Oliveira, Fabio Benevenuti, Luis A. C. Benites, and Fernanda L. Kastensmidt are with the Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre 90040-060, Brazil (e-mail: adria.oliveira@inf.ufrgs.br; fabio.benevenuti@inf.ufrgs.br; luis.benites@inf.ufrgs.br; fglima@inf.ufrgs.br).

Lucas A. Tambara was with Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre 90040-060, Brazil. He is now with Cobham Gaisler AB, 411 19 Göteborg, Sweden (e-mail: lucas.a.tambara@gaisler.com).

Nemitala Added, Vitor A. P. Aguiar, and Nilberto H. Medina are with Universidade de São Paulo (USP), São Paulo 05508-220, Brazil (e-mail: nemitala@if.usp.br; vpaguiar@if.usp.br; medina@if.usp.br).

Marcilei A. G. Silveira is with the Centro Universitário da FEI, São Bernardo do Campo 09850-90, Brazil (e-mail: marcilei@fei.edu.br).

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TNS.2020.2995729

implemented in an SRAM-based FPGA, 2) exploring mitigation techniques to improve the overall system reliability, and 3) analyzing the impact of the processor speed in the error rate. Our results reveal that configuration memory scrubbing combined with periodical reset shows up to be the most cost trade-off efficient method. The results obtained also show that not only the processor core must be protected but also additional techniques aiming at its peripherals and cache memories may be required to cope with data errors and timeouts.

II. Problems of Using Soft Core Processors Implemented Into COTS SRAM-Based FPGAs in SEE-Prone Environments

As mentioned in Section I, the primary concern when implementing soft core processors into COTS SRAM-based FPGAs is the processor vulnerability to persistent errors in the configuration memory, which may affect the core functionality, and errors in the embedded memories, which may cause incorrect computations.

The literature describes a plethora of fault-tolerant techniques to improve the reliability of FPGA-based designs, such as soft core processors. The main techniques are based on hardware redundancy. They are implemented at design level and usually provide two or more instances of the hardware component. One of the most well known is the triple modular redundancy (TMR), which consists of triplicating a circuitry and vote the outputs [7], [8]. The drawback of redundancy-based techniques such as TMR is the amount of resources required to implement the mitigation. In addition, these techniques do not clear out persistent errors in the configuration memory of an SRAM-based FPGA but only mask them. As a consequence, additional techniques are required to maintain the configuration memory consistent, such as scrubbing. Periodic scrubbing is, in fact, the only technique that repairs the logic and corrects bit-flips in the configuration memory [9]. However, system reset may also be required to re-establish the system to a known state. Watchdogs and checkers are also usually used to verify the hardware integrity and detect control-flow errors [10], and error correction codes (ECCs) are commonly applied to detect and correct errors in embedded memories. The tolerance improvement provided by hardware techniques comes at the expense of power consumed and dissipated by the extra hardware. Many works have validated the reliability improvement of such hardware-based techniques applied to soft processors embedded into SRAM-based FPGAs, such as the Leon3 [11] and Cortex-M0 [12].

At higher levels such as software, software-implemented hardware fault tolerance (SIHFT) are techniques that deal with faults affecting the hardware by protecting only the software, without hardware modification [13]. The software-based methods may also rely on adding redundancy and comparison for error detection. Performance degradation is the main drawback of SIHFT techniques due to the additional instructions in the code.

As one can notice, developing a soft core processor-based system into a COTS SRAM-based FPGA consists of engineering not only a system that is capable of providing high reliability, but also a balanced trade-off in terms of resources usage, performance, and power consumption. As discussed, any fault-tolerant technique adopted will impose overheads in at least one of these parameters.

III. Case Study: RISC-V Processor

The RISC-V ISA is a standard open architecture, highly extensible, and flexible that allows the development of general-purpose computing systems [14]. The standard was established for the design implementation not to be architecture- or technology-dependent. Processors based on the RISC-V ISA have been adopted in very diverse application fields as a consequence of its advantages. The work [15] discusses the emerged interest in using RISC-V processors not only in terrestrial but also in future space applications. At the same time, the use of COTS devices in space missions has gained popularity mainly due to performance requirements [16]. The same trend is applied to COTS FPGAs. Concerning the problematic described in the last section, this work aims to evaluate the susceptibility to soft errors of a RISC-V architecture-based processor embedded into a COTS SRAM-based FPGA.

Several works have addressed fault-tolerant mechanisms to harden RISC-V processors. Most approaches are based on redundancy techniques and ECC. A hybrid fault-tolerant architecture based on redundancy and ECC techniques and capable of providing single error correction and double error detection was presented in [17]. The author observed through FI experiments that bit-flips in the pipeline are more likely to lead to errors in the register file. In addition, the author presented results concerning the implementation of the design, such as the fact that although the fault tolerance has been met, the implemented design did not meet the expected clock frequency efficiency and the maximum resources usage requirement. These results are consequences of the long paths inherent to a redundant design.

Gupta *et al.* [18] proposed a new fault-tolerant RISC-V microprocessor architecture based on space and time redundancy techniques to harden the arithmetic logic unit (ALU) and ECC to protect the registers and memories. The reliability of the system in the presence of single-bit faults was evaluated through FI. The results presented show that the proposed scheme is able to harden the ALU and data path against soft and hard errors with a penalty of 20% in area and 25% in performance. Cho [19] compared the soft error susceptibility affecting flip-flops (FFs) of two RISC-V soft core processors, rocket and Berkeley out-of-order machine (BOOM). The results demonstrated that the most vulnerable parts of RISC-V cores are the control status register (CSR) and register file. The authors also showed that although the raw error rates from the two processor cores are different, the error rates depending on the applications have strong correlations. A method to protect the register file of a RISC-V soft core processor (Rocket) based on a design tool-based inferred redundancy and parity-based error detection was proposed in [20]. The authors showed through FI by emulation the effectiveness of the scheme

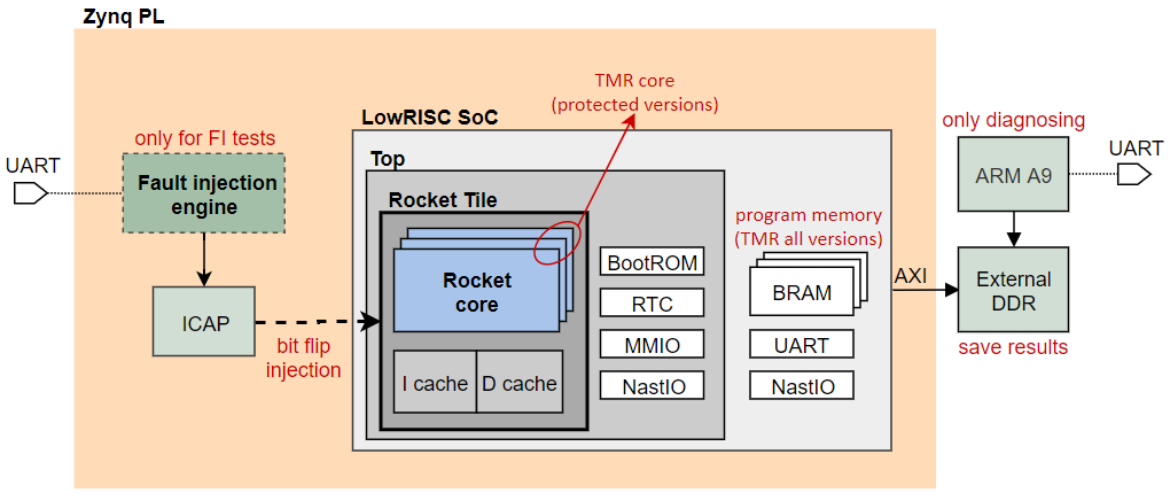

Fig. 1. LowRISC SoC implementation into Zynq APSoC and ZebBoard development board.

proposed in correcting single-bit errors at a considerably lower cost than a traditional TMR approach.

The main contribution of our work is to explore not only the susceptibility of a RISC-V soft core processor design but also the SRAM-based FPGA configuration memory sensitivity and how it affects the entire system under heavy ion-induced SEEs. We selected the general-purpose RISC-V Rocket processor as a case study, which is a customized scalar soft core generated using the open-source Rocket Chip SoC generator [4]. The processor has a 5-stage in-order pipeline, L1 data and instruction caches, integer ALU, and an optional floating-point unit (FPU). This work is based on the lowRISC SoC implementation that contains the Rocket Chip generator; peripherals, such as universal asynchronous receiver/transmitter (UART) and memory management controller (MMC); an advanced extensible interface (AXI) compatible to the DDR memory controller from Xilinx; and an adapted Linux kernel [21]. A simplification of the lowRISC SoC design implemented is described in Fig. 1. The Rocket Tile is the block that encloses the RISC-V Rocket core and the L1 caches. Our evaluation consists of a single-core system with 8 KB L1 data and instruction caches and no L2 cache and FPU. The L1 caches are implemented in block random access memories (BRAMs). This work focuses on the reliability analysis of the Rocket core under faults. We also evaluate the effectiveness of mitigation techniques applied only in the processor core. This work does not cover the protection of the cache memories, buses, and peripherals in the system.

IV. Evaluation Methodology

The case-study device adopted for the proposed analysis is the COTS Xilinx Zynq-7000 (XC7Z020 part) All-Programmable SoC (APSoC). The device, which is fabricated in a 28-nm CMOS process, is composed of two main hardware blocks: the programmable logic (PL), which consists of an SRAM-based FPGA layer, and the processing system (PS), which is formed around an ARM Cortex-A9 processor [22].

Fig. 1 presents the system setup, which is based on a commercially available ZedBoard Development Board, the lowRISC SoC implemented into the Zynq PL, the ARM A9, and the external DDR memory. The design under test (DUT) is the Rocket core, which executes a benchmark and saves the outputs in the DDR. The diagnosing is performed by the ARM A9 processor that computes the golden output values, accesses the DDR memory, and compares both Rocket and golden results. SDC and timeout events are the functional failures measured from the results. SDC refers to wrong computations in the benchmark without system interruption. A timeout refers to the absence of benchmark outputs after a given amount of time, which can occur due to SEFIs or processor hangs.

The system assessment is performed using unhardened and protected Rocket core versions. The program memory is triplicated in all designs, including the unhardened, to avoid the processor to boot with corrupted instructions. We submitted the system to FI and heavy-ions irradiation tests. The following sections present details about the benchmarks, characterize the DUT versions tested, describe both experimental setups, and list the adopted metrics to assess the system.

A. Software Application

Three benchmark applications are used for the DUT evaluation: 32 × 32 matrix multiplication (MxM), advanced encryption standard (AES) with 32 elements array, and Quicksort (Qsort) with 255 elements array. The applications are standard benchmarks extensively used in tests of microprocessors and FPGA designs under radiation [23]. The execution is in bare metals, which means the benchmark runs in a processor without an operating system. The application data are 32-bit length and encoded in fixed point (Q16). All the benchmarks are initialized with data randomly generated through a pseudorandom number generator (PRNG). 16-bit size checksums are computed for the data result of MxM and AES applications to minimize the verification time, whereas

TABLE I
BENCHMARK APPLICATION CHARACTERISTICS

Benchmark	Array length (32-bits data)	Workload (bits)	Clock cycles (#)	Verification (data check)
MxM	32x32	32,768	14,171,672	16-bit checksum
AES	32	1,024	1,170,055	16-bit checksum
Qsort	255	8,160	10,554,246	full array

TABLE II
DETAILED RESOURCE USAGE FOR UNHARDENED, CGTMR, AND FDTMR RISC-V CORE DESIGNS AND THE RESPECTIVE LOWRISC SOC WITH THE PERCENTAGE OF ZYNQ PL UTILIZATION

Design		LUT (%)	FF (%)	Carry	BRAM (%)	DSP (%)
Unhard	RISC-V	4,457 (8%)	1,701 (2%)	175	0	4 (2%)
	SoC	17,329 (32%)	12,164 (11%)	544	68 (49%)	9 (4%)
CGTMR	RISC-V	26,556 (49%)	11,036 (10%)	0	0	0
	SoC	39,232 (74%)	21,495 (20%)	369	68 (49%)	5 (2%)
FDTMR	RISC-V	29,706 (56%)	9,756 (9%)	0	0	0
	SoC	42,382 (80%)	20,215 (19%)	369	68 (49%)	5 (2%)

the entire array is compared in the Qsort. Table I summarizes the benchmark characteristics.

B. Design Under Test

The unhardened core design consists of the unprotected RISC-V hardware, whereas TMR is applied in the protected versions of the core. Two TMR granularities are tested: Coarse Grain TMR (CGTMR) and fine grain distributed TMR (FDTMR). The CGTMR consists of triplicating the entire core and voting the outputs bit by bit, whereas in the FDTMR the submodules are triplicated. The netlists hardened by TMR are generated by an automated tool based on the Cadence's EDA flow [24]. The voters are also triplicated to avoid single points of failure. Other lowRISC SoC elements remained unprotected since we consider only the Rocket core as the DUT.

The built-in Xilinx scrubbing [25] running at 50 MHz is enabled to clean bit-flips in the FPGA configuration memory (only versions labeled with *_S*). A watchdog timer, controlled by the ARM A9, reprograms the FPGA whenever a timeout occurs (only versions labeled with *_T*). Design versions with both scrubbing and watchdog enabled are labeled with *_ST*. A processor reset is performed in all designs to return the system to a known state after each benchmark execution. All designs run at 20 MHz, which was the maximum frequency achieved in the TMR implementations.

Table II presents the FPGA resource usage of the RISC-V core versions implemented (unhardened, CGTMR, and FDTMR). The table also presents the resource usage of the lowRISC SoC for each core version and the percentage of utilization of the Zynq PL. The use of the built-in Xilinx scrubber and the watchdog, which is performed by the ARM A9, does not impact the resource usage when adopted and therefore are not present in the table. The full Rocket FDTMR SoC required more slice lookup table (LUT) resources (i.e., 55 041) than are available in the Zynq PL (i.e., 53 200). As a result, we decided to allow design optimization to be performed by the Vivado tool during the implementation phase. The resource usage of the FDTMR design presented in Table II is after the optimization. The netlists of the TMR designs are processed using an automatic tool. Therefore, all the arithmetic and logic operations are implemented in LUTs, and the TMR core versions do not use any DSP or carry chain.

In the FI experiments, the unhardened and CGTMR versions of the Rocket core are tested running the MxM, AES, and Qsort benchmarks. The FDTMR design is not executed because its version and the FI engine combined require more LUT resources than those available in the Zynq PL. As described in Section IV-C, the FI engine uses the FPGA Internal Configuration Access Port (ICAP) port to inject faults. Therefore, the scrubbing is disabled since it also uses the ICAP port, and no parallel accesses are allowed by the interface.

In the heavy-ions tests, all Rocket core versions are tested running the MxM benchmark. The unhardened processor at 50 MHz of the clock frequency is also tested to assess the impact of the processor speed on the system reliability.

C. Fault Injection Setup

In this work, we use an FI engine based on the work performed in [26] that focuses on Xilinx's 7 Series FPGAs and APSoCs. The FI engine explores the ICAP hardware module to read and write configuration memory frames while XOR'ing the value of specific bits. The controller software running on a host computer determines the position of each injected bit-flip. The FI engine injects random faults in the target FPGA area in order to mimic bit-flips caused by ionizing radiation. The FI setup is also shown in Fig. 1.

Although it is feasible to define a restricted floorplanning area to individual FPGA design blocks, isolating the Rocket core from the L1 cache memory is not synthesizable. Thus, the entire Rocket Tile area is affected by the FI. To normalize the FI among all designs, the defined target area size is the same for both unhardened and CGTMR designs. One bit-flip is injected at a time. After each injection, the Rocket processor is logically reset, and the application execution re-starts. Faults are accumulated in the design until a functional failure (i.e., SDC or timeout) being detected. The number of faults accumulated until that point is recorded, and then the FPGA is reprogrammed with the fault-free design. This procedure is repeated until 1000 functional failures are collected.

D. Heavy-Ions Setup

The device adopted for this work had its package thinned to ensure the ion penetration into the active region of the silicon during heavy ions experiments. The total thickness of the Zynq PL's passive layers is 12.87 μm [27].

The irradiation tests were conducted through accelerated heavy ions in the 8UD Pelletron accelerator facility at Laboratório Aberto de Física Nuclear of the Universidade de São Paulo (LAFN-USP), Brazil [28]. The device was irradiated in-vacuum using oxygen (^{16}O) and carbon (^{12}C) ions at normal incidence. The effective linear energy transfer (LET) and the penetration of the particles are described in Table III. The regular beam fluxes ranged from 2.85×10^2 up to 5.28×10^2 p/cm^2/s, being the tests fluence in the order of 10^6 p/cm^2. We estimated through static tests that the chosen particle fluxes

produce an average error rate of 5 bit-flips/s in the Zynq PL. The built-in Xilinx scrubbing scans the entire configuration memory in 16 ms, and the latency for the correction of a single error is of about 19 ms [25]. As a consequence, we assume that the scrubbing rate is capable of correcting all single upsets induced by the chosen particle fluxes.

E. Evaluated Metrics

1) Empirical Reliability: The empirical reliability is defined as the capability of the design to provide the correct result in time. The reliability analysis as a function of fluence is proposed in [29], instead of using it from the time domain directly. Different from [29], our reliability function $R(\phi)$ is defined using the empirical data results (i.e., number of SDCs and timeouts) from the experiments, as represented in (1). The $F(\phi)$ is the cumulative distribution function related to the number of accumulated faults or heavy ions fluence

$$R(\phi) = 1 - F(\phi). \tag{1}$$

2) Cross Section: The cross section is a standard metric used to evaluate the radiation-sensitive area of the device. As defined in (2), the cross section (σ) is computed by the relation between the number of functional failure (i.e., number of SDCs and timeouts) and the total heavy ions fluence

$$\sigma = \frac{\#\text{errors}}{\text{fluence}}. \tag{2}$$

3) Mean Workload Between Failures: Mean workload between failures (MWBF) is a metric generally used to evaluate the system reliability. The MWBF analyzes the trade-off between the error rate and performance by the amount of data correctly computed until an output error occurs [30]. Equation (3) relates the workload to cross section, beam flux of the heavy-ions experiment, and the application execution time (in seconds). Higher MWBF means higher operational system reliability

$$\text{MWBF} = \frac{\text{workload}}{(\sigma \times \text{flux} \times \text{exec.time})}. \tag{3}$$

V. Results Obtained

A. Fault Injection Results

Fig. 2 presents the empirical reliability curves obtained for each DUT. Single faults were the main cause that leads the unhardened Rocket core to achieve a reliability of less than 88%. In [31], 94.6% of the results are correct, on average. Considering different soft core processor architectures, Azambuja *et al.* [32] obtained a correct processing rate of different benchmark applications of around 91% with an MIPS soft core processor, whereas in [12] achieved reliability of about 97% for single faults with an unhardened ARM Cortex-M0 soft core processor. Since our FI affects only the Rocket Tile area, our results are pessimistic and consider the worst case scenario in which a single fault has a high probability of provoking a failure in the execution flow.

The obtained results concerning the unhardened designs demonstrated that, on average, 79% of the functional failures

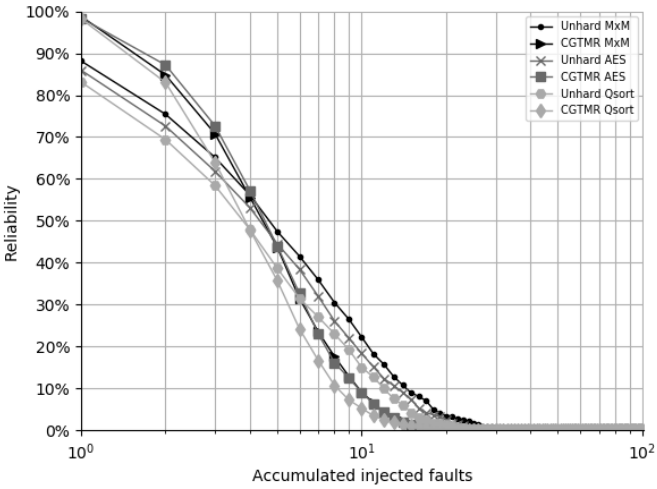

Fig. 2. Empirical reliability obtained from the FI by emulation test campaigns with the RISC-V designs running MxM, AES, and Qsort benchmarks at 20 MHz.

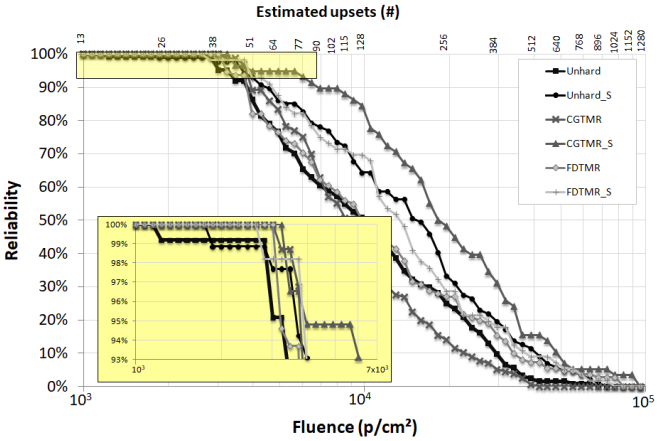

Fig. 3. Empirical reliability obtained from the heavy-ions test campaign with the Rocket designs running MxM benchmark at 20 MHz.

are timeouts, and 21% are SDCs. With regard to the fully triplicated designs, the failure ratio is divided into 8% SDCs and 92% timeouts. As expected, the CGTMR presented higher reliability in the presence of single faults. However, the TMR masking capability is overcome when the design is affected by a higher number of accumulated faults due to its higher exposed area. Nonetheless, the CGTMR did not achieve the maximum reliability for single faults because the bit-flips injected covered the full Rocket Tile area and not only the triplicated core.

B. Heavy-Ions Results

Fig. 3 presents the empirical reliability curves for the designs tested using ^{16}O ions. As one can see, the unhardened Rocket maintained maximum reliability until fluence of around 1.15×10^3 p/cm^2. The configuration memory cross section per device of the Zynq PL is 1.28×10^{-2} cm^2 [27]. From (2), the estimated number of upsets in the configuration memory is defined by cross section multiplied by the fluence. Therefore, the unhardened core is 100% reliable until 15 bit-flips accumulated in the configuration memory and its reliability drops to 90% with 50 upsets (i.e., fluence of 3.98×10^3 p/cm^2).

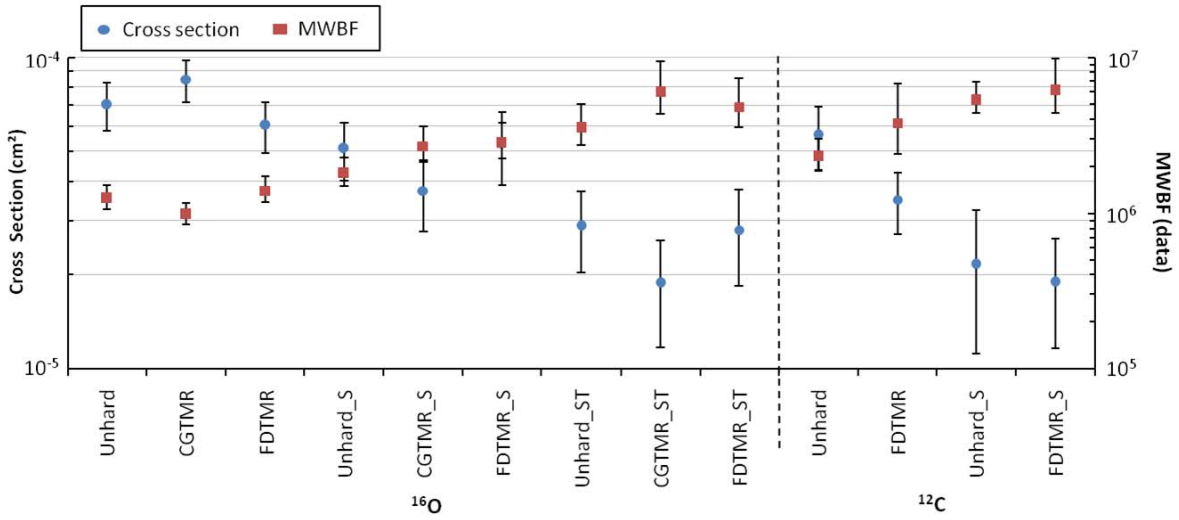

Fig. 4. Total dynamic cross section and MWBF results from heavy-ions experiments for the Rocket designs running MxM benchmark at 20 MHz.

The design with TMR and scrubbing presented a reliability three times higher than the unhardened design. The *CGTMR_S* design sustained a high level of reliability until 3.24×10^3 p/cm^2, which represents 41 estimated bit-flips in the configuration memory. However, these upsets are not accumulated since the scrubbing is enabled. All designs are able to maintain the correctness for more bit-flips than in the FI tests due to the target area of faults. The FI focuses the injections in an active area of the design, whereas the full FPGA is affected by the heavy ions, and most of the upsets might occur in an unused area.

Table III describes the dynamic cross section results for SDCs and timeouts. As discussed, bit-flips in the configuration memory may change the circuitry functionality leading to functional failures. For instance, faults in the control flow may provoke processor exceptions or losses of sequence. Bit-flips affecting the instruction cache or cache controller can also lead to hangs. Moreover, timeouts may occur when the clock tree or the system reset is affected by a soft error. Faults affecting data memory elements more likely cause the SDCs, and the processor register file is implemented in LUTs, which increases the susceptibility.

Fig. 4 shows the results in terms of dynamic cross section and MWBF. A reliable design must present a lower cross section and, therefore, a higher MWBF. One can notice modest improvements in the dynamic cross section when only scrubbing or TMR is applied. In addition, the absolute number of the CGTMR cross section is slightly higher than the single unprotected core. TMR designs are more prone to bit-flips because of the significant number of resources used, and the masking effect is compromised with the accumulation of faults. In contrast, the cross section can be reduced up to three times and the MWBF is improved more than two times when TMR and scrubbing are combined in a design. Surprisingly, the FDTMR results are relatively less expressive than expected. That might be a consequence of the oversimplification of the design during the synthesis optimization phase. As mentioned above, some of the optimization attributes were not disabled due to the limited resources of the Zynq PL. The designs with scrubbing and watchdog (*_ST* versions) presented a reduction of more than one order of magnitude in the timeout cross section. Moreover, the MWBF is improved almost three times. Instead of using a processor to monitoring timeouts, a better approach is implementing a dedicated register-transfer level (RTL) module that consumes few resources not impacting in area.

TABLE III
DYNAMIC CROSS SECTION RESULTS FOR SDCS AND TIMEOUTS FROM HEAVY-IONS EXPERIMENTS

Beam*	Design	Fluence (p/cm^2)	SDC σ (cm^2)	Timeout σ (cm^2)
^{16}O 6.7 23 μm	Unhard	1.76×10^6	3.46×10^{-5}	3.57×10^{-5}
	CGTMR	1.84×10^6	2.88×10^{-5}	5.59×10^{-5}
	FDTMR	1.83×10^6	2.73×10^{-5}	3.33×10^{-5}
	Unhard_S	1.71×10^6	2.34×10^{-5}	2.75×10^{-5}
	CGTMR_S	1.56×10^6	1.41×10^{-5}	2.31×10^{-5}
	FDTMR_S	1.06×10^6	2.27×10^{-5}	3.02×10^{-5}
	Unhard_ST	1.56×10^6	2.69×10^{-5}	1.92×10^{-6}
	CGTMR_ST	1.44×10^6	1.81×10^{-5}	6.96×10^{-7}
	FDTMR_ST	1.18×10^6	2.63×10^{-5}	1.70×10^{-6}
^{12}C 3.7 36 μm	Unhard	1.31×10^6	3.50×10^{-5}	2.13×10^{-5}
	FDTMR	1.20×10^6	1.58×10^{-5}	1.91×10^{-5}
	Unhard_S	1.38×10^6	1.09×10^{-5}	1.09×10^{-5}
	FDTMR_S	1.37×10^6	1.09×10^{-5}	8.00×10^{-6}

*Beam description: ion; LET in $MeV/mg/cm^2$; and penetration.

Fig. 5 shows the dynamic cross section and MWBF results for the unhardened Rocket designs running the MxM benchmark at 20 and 50 MHz. Increasing the clock frequency improves the MWBF while slightly affecting the system cross section. The higher the processor speed, the faster the application is executed, and consequently, more data is computed correctly before experiencing a failure. However, higher clock

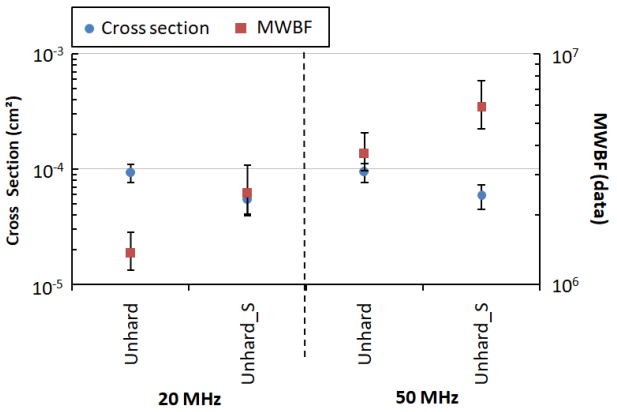

Fig. 5. Total dynamic cross section and MWBF results from heavy-ions experiments for the unhardened Rocket design running MxM benchmark at 20 and 50 MHz.

frequencies might affect the cross section significantly. Therefore, the trade-off cross section and performance must be considered.

C. Discussion

The results obtained from FI and heavy ions show that the use of scrubbing and periodic reset are essential for SRAM-based FPGA designs, as expected. Scrubbing prevents the accumulation of bit-flips in the configuration memory. The reset restores the system to a known state. However, the results demonstrated that the unhardened Rocket core is highly susceptible to single bit-flips, and additional user-level mitigation techniques are required to mask or correct faults. Although combining TMR and scrubbing was supposed to increase the fault tolerance of the system, only modest improvements were observed in the results. The main reason relies on the fact that only the processor core was protected, and essential parts of the SoC were still vulnerable to soft errors, such as the caches and peripherals. Cache memories are highly sensitive elements in processors. As demonstrated in [27], errors in unprotected L1 caches can highly impact the processor cross section. Methods like parity checks can be applied to the caches to increase reliability. The peripherals and buses should also be triplicated in order to avoid single points of failure. We believe that errors in the unprotected caches and peripherals masked the real improvement of the TMR Rocket core.

The normalized sensitivity for distinct soft processors is described in [33]. The Cortex-M0 presented the lowest sensitivity (i.e., 3.79%), and the Picoblaze the highest (i.e., 6.11%), whereas the Leon3 sensitivity is 4.81%. The estimated Rocket (lowRISC) sensitivity is 6.86% [31], which indicates that the Rocket might be more prone to faults than other cores. Table IV shows radiation test results for soft core processors embedded into SRAM-based FPGAs. Although only the Rocket and Cortex-M0 results are directly related, for the sake of comparison, one can observe the cross section improvements in adding mitigation methods to the system. Despite the fact that the Cortex-M0 has the lowest sensitivity [33], the heavy-ions results (Table IV) show the Rocket core with a lower cross section than the Cortex-M0. However, the benefits of TMR and scrubbing are more visible in the latter. In [12], both the Cortex-M0 core and also all the external elements are triplicated, which validates the importance in protecting all resources. A pre-eminent cross section improvement for Leon3 is presented in [11], in which the entire SoC is triplicated. Additionally, when the BRAM memory scrubbing is applied, it demonstrates an improvement of 50×. Due to the lower error rate of neutrons experiments, the scrubber is able to clean the configuration memory effectively, while avoiding many accumulated upsets at the same time. When combined with the masking factor of TMR, the scrubber leads to a high impact on the cross section reduction.

TABLE IV
CROSS SECTION COMPARISON OF SOFT PROCESSORS IMPLEMENTED IN SRAM-BASED FPGAS

Soft-processor	Experiment	Cross section (cm^2)	
		Unhard	TMRbest+Scrub (Improvement)
Rocket (lowRISC SoC)	Heavy Ions - Zynq-7000	7.03×10^{-5}	1.89×10^{-5} (3×)
Cortex-M0 [12]	Heavy Ions - Zynq-7000	1.14×10^{-4}	9.19×10^{-5} (4.5×)
Leon3 [11]	Neutrons - Kintex-7	2.61×10^{-9}	9.68×10^{-11} (27×)

The FI results of the Rocket processor presented in [19] demonstrate that, on average, 3.5% of single upsets lead to SDCs, whereas 3.4% and 8% result in hangs and unexpected terminations, respectively. In [31], single faults lead to a hang rate of around 2.6% among the benchmarks, and SDCs can be as lower as 0.19% up to 3.07% depending on the execution time. Despite the structural differences in the FI methods, which difficult straightforward comparison, one can notice that the Rocket processor is more prone to faults that crash or lock the core, but also it presents a high SDC rate, which is also confirmed by our results. The understanding of the error distribution and how it affects the processor execution is essential to implement the most suitable mitigation techniques. As demonstrated in [19], the most critical faults are the ones affecting the register file, CSR, and the integer pipeline of the Rocket processor. Thus, the designer can apply mitigation focusing on these elements, such as ECC on the register file, and triplicating the CSR and pipeline. Alternatively, a lockstep method can be implemented for the RISC-V architecture enhancement, similar to the lockstep mode of ARM Cortex R5 [34]. However, as indicated by our results, protecting only the core is not enough to ensure the high reliability of the entire system. Fault mitigation and correction methods must be extensively applied to all elements of the SoC, and the more heterogeneous they are, the better.

Therefore, a better approach for SoCs with complex soft core processors, such as the RISC-V Rocket core, might be applying a heterogeneous hybrid solution that mixes hardware and software protection. At the hardware level, a dedicated RTL watchdog module can be used to monitor timeouts and reprogram the FPGA when required. The processor core enhancement may rely on TMR or lockstep, and the peripherals should also be triplicated. ECC or parity method may be used to protect the register file and memories. FPGA scrubbing and periodic reset must be performed to avoid the accumulation of faults and reestablish the system state,

respectively. At the software level, a SIHFT technique can be applied to lead with SDCs. Moreover, increasing the processor speed is an alternative to achieve better MWBF, without affecting the cross section significantly.

The RISC-V-based processors are already being used in many terrestrial applications, and there is an expectation in adopting the RISC-V also in space systems [15], [16]. The primary outcome from our work is the demonstration that a RISC-V processor embedded into a COTS SRAM-based FPGA presents analogous SEE sensitivity as the state-of-the-art soft processors, and the system enhancement through fault-tolerant techniques may significantly improve the reliability. Our results also indicate that the processor core is not the only critical element in the system that requires protection, and the hardening of caches and peripherals is also essential to increase the overall reliability of the system.

VI. Conclusion

This article assessed the soft error vulnerability of an in-order single-issue soft core RISC-V Rocket processor implemented in a COTS SRAM-based FPGA. As a case study, we embedded the Rocket processor into the Xilinx Zynq APSoC. Accelerated irradiation tests demonstrated that the RISC-V processor implemented in an SRAM-based FPGA is particularly susceptible to functional failures caused by upsets in the configuration memory. The unhardened Rocket core achieved a maximum of 88% of reliability in the FI experiments. Scrubbing combined with periodic reset is the most cost trade-off efficient approach tested. Due to the complexity of the lowRISC SoC, not only the Rocket core but also its caches and peripherals should be protected with proper solutions. Such solutions should address the specific vulnerabilities of each component to improve the overall system reliability while maintaining the trade-off with performance.

References

[1] C. Gaisler. (Apr. 2020). *GRLIB IP Core User's Manual, Version 2020.1*. [Online]. Available: https://www.gaisler.com/products/grlib/grip.pdf

[2] J. Yiu, *System-on-Chip Design With Arm Cortex-M Processors*. Cambridge, U.K.: ARM Education Media, 2019.

[3] *MicroBlaze Micro Controller System v3.0, LogiCORE IP Product Guide*, Doc. PG116, Xilinx, Nov. 2019.

[4] K. Asanović *et al.*, "The rocket chip generator," Dept. Elect. Eng. Comput. Sci., Univ. California, Berkeley, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2016-17, Apr. 2016.

[5] M. Wirthlin, "High-reliability FPGA-based systems: Space, high-energy physics, and beyond," *Proc. IEEE*, vol. 103, no. 3, pp. 379–389, Mar. 2015.

[6] H. Quinn, "Challenges in testing complex systems," *IEEE Trans. Nucl. Sci.*, vol. 61, no. 2, pp. 766–786, Apr. 2014.

[7] F. L. Kastensmidt, L. Sterpone, L. Carro, and M. S. Reorda, "On the optimal design of triple modular redundancy logic for SRAM-based FPGAs," in *Proc. Design, Automat. Test Eur.*, vol. 2, Mar. 2005, pp. 1290–1295.

[8] H. Quinn, K. Morgan, P. Graham, J. Krone, M. Caffrey, and K. Lundgreen, "Domain crossing errors: Limitations on single device triple-modular redundancy circuits in Xilinx FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 54, no. 6, pp. 2037–2043, Dec. 2007.

[9] I. Herrera-Alzu and M. Lopez-Vallejo, "Design techniques for Xilinx Virtex FPGA configuration memory scrubbers," *IEEE Trans. Nucl. Sci.*, vol. 60, no. 1, pp. 376–385, Feb. 2013.

[10] A. Mahmood and E. J. McCluskey, "Concurrent error detection using watchdog processors—A survey," *IEEE Trans. Comput.*, vol. C-37, no. 2, pp. 160–174, Feb. 1988.

[11] A. M. Keller and M. J. Wirthlin, "Benefits of complementary SEU mitigation for the LEON3 soft processor on SRAM-based FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 64, no. 1, pp. 519–528, Jan. 2017.

[12] L. A. C. Benites *et al.*, "Reliability calculation with respect to functional failures induced by radiation in TMR arm cortex-M0 soft-core embedded into SRAM-based FPGA," *IEEE Trans. Nucl. Sci.*, vol. 66, no. 7, pp. 1433–1440, Jul. 2019.

[13] O. Goloubeva, M. Rebaudengo, M. S. Reorda, and M. Violante, *Software-Implemented Hardware Fault Tolerance*. New York, NY, USA: Springer, 2006.

[14] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, "The RISC-V instruction set manual, user-level ISA, version 2.0," Dept. Elect. Eng. Comput. Sci., Univ. California, Berkeley, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2014-54, May 2014, vol. 1.

[15] S. Di Mascio, A. Menicucci, E. Gill, G. Furano, and C. Monteleone, "Leveraging the openness and modularity of RISC-V in space," *J. Aerosp. Inf. Syst.*, vol. 16, no. 11, pp. 454–472, Nov. 2019.

[16] S. Esposito *et al.*, "COTS-based high-performance computing for space applications," *IEEE Trans. Nucl. Sci.*, vol. 62, no. 6, pp. 2687–2694, Dec. 2015.

[17] W. F. Heida, "Towards a fault tolerant RISC-V softcore," M.S. thesis, Fac. Elect. Eng., Math. Comput. Sci., Delft Univ. Technol., Delft, The Netherlands, 2016.

[18] S. Gupta, N. Gala, G. S. Madhusudan, and V. Kamakoti, "SHAKTI-F: A fault tolerant microprocessor architecture," in *Proc. IEEE 24th Asian Test Symp. (ATS)*, Nov. 2015, pp. 163–168.

[19] H. Cho, "Impact of microarchitectural differences of RISC-V processor cores on soft error effects," *IEEE Access*, vol. 6, pp. 41302–41313, 2018.

[20] A. Ramos, A. Ullah, P. Reviriego, and J. A. Maestro, "Efficient protection of the register file in soft-processors implemented on Xilinx FPGAs," *IEEE Trans. Comput.*, vol. 67, no. 2, pp. 299–304, Feb. 2018.

[21] lowRISC. *Untethered lowRISC Tutorial*. Accessed: Aug. 2019. [Online]. Available: https://www.lowrisc.org/docs/untether-v0.2

[22] *Zynq-7000 All Programmable SoC Technical Reference Manual, v1.11*, Doc. UG585, Xilinx, Sep. 2016.

[23] H. Quinn *et al.*, "Using benchmarks for radiation testing of microprocessors and FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 62, no. 6, pp. 2547–2554, Dec. 2015.

[24] L. A. C. Benites and F. L. Kastensmidt, "Automated design flow for applying triple modular redundancy (TMR) in complex digital circuits," in *Proc. IEEE 19th Latin-Amer. Test Symp. (LATS)*, Mar. 2018, pp. 230–233.

[25] *Soft Error Mitigation Controller v4.1, LogiCORE IP Product Guide, Vivado Design Suite*, Doc. PG036, Xilinx, Apr. 2018.

[26] J. Tonfat, L. Tambara, A. Santos, and F. Kastensmidt, "Method to analyze the susceptibility of HLS designs in SRAM-based FPGAs under soft errors," in *Applied Reconfigurable Computing*, V. Bonato, C. Bouganis, and M. Gorgon, Eds. Cham, Switzerland: Springer, 2016, pp. 132–143.

[27] L. A. Tambara *et al.*, "Heavy ions induced single event upsets testing of the 28 nm Xilinx Zynq-7000 all programmable SoC," in *Proc. IEEE Radiat. Effects Data Workshop (REDW)*, Jul. 2015, pp. 29–34.

[28] V. A. P. Aguiar *et al.*, "Experimental setup for single event effects at the São Paulo 8UD pelletron accelerator," *Nucl. Instrum. Meth. Phys. Res. B, Beam Interact. Mater. At.*, vol. 332, pp. 397–400, Aug. 2014.

[29] M. Berg, K. LaBel, M. Campola, and M. Xapsos, "Analyzing system on a chip single event upset responses using single event upset data, classical reliability models, and space environment data," NASA, Washington, DC, USA, Tech. Rep., 2017. [Online]. Available: https://ntrs.nasa.gov/archive/nasa/casi.ntrs.nasa.gov/20170009889.pdf

[30] P. Rech, L. L. Pilla, P. O. A. Navaux, and L. Carro, "Impact of GPUs parallelism management on safety-critical and HPC applications reliability," in *Proc. 44th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Jun. 2014, pp. 455–466.

[31] A. Ramos, J. A. Maestro, and P. Reviriego, "Characterizing a RISC-V SRAM-based FPGA implementation against single event upsets using fault injection," *Microelectron. Rel.*, vol. 78, pp. 205–211, Nov. 2017.

[32] J. R. Azambuja *et al.*, "A fault tolerant approach to detect transient faults in microprocessors based on a non-intrusive reconfigurable hardware," *IEEE Trans. Nucl. Sci.*, vol. 59, no. 4, pp. 1117–1124, Aug. 2012.

[33] N. A. Harward, M. R. Gardiner, L. W. Hsiao, and M. J. Wirthlin, "Estimating soft processor soft error sensitivity through fault injection," in *Proc. IEEE 23rd Annu. Int. Symp. Field-Programmable Custom Comput. Mach.*, May 2015, pp. 143–150.

[34] *Cortex-R5 and Cortex-R5F Technical Reference Manual. Rev:r1p1*, document, 2011. [Online]. Available: http://infocenter.arm.com/help/topic/com.arm.doc.ddi0460c/DDI0460C_cortexr5_trm.pdf